

Calibration-Free Stereo Depth via a Trainable Anisotropic γ -Grid

A Computer Vision Project Report

Roman Sahakyan

2026

Abstract

We present a stereo depth estimation pipeline that replaces the classical pinhole intrinsic matrix with a learnable anisotropic γ -grid, trained from a handful of in-scene click pairs whose ground-truth distances are known. The system requires no calibration target (no checkerboard), runs in pure NumPy with hand-derived analytical gradients, and trains in well under a second on a CPU. We benchmark it against a classical pinhole-OLS baseline on 43 tape-measured points spanning 1.35–8.75 m. Under leave-one-out cross-validation the γ -grid plus a 3-parameter quadratic post-correction reduces mean absolute depth error from 0.373 m (pinhole) to 0.269 m, a 28% improvement; it cuts the maximum single-point error from 2.56 m to 1.85 m. We document an honest overfitting gap between in-sample and held-out evaluation and a mirror tradeoff between the two methods on near versus far points, both of which motivate concrete future work.

1 Problem and Motivation

The textbook recipe for stereo depth from two consumer cameras starts with intrinsic calibration via Zhang’s algorithm: roughly twenty checkerboard images per camera are required, and the result still assumes a single radial-tangential distortion model per camera. We pose the inverse question: *given a stereo pair with unknown intrinsics, can we recover metric depth using only a handful of in-scene click pairs whose ground-truth distance is known?*

Our setting is operationally simple. The user supplies ≥ 15 calibration clicks of the form $(u_L, v_L, u_R, v_R, Z_{\text{true}})$, where Z_{true} is measured with a tape; afterwards the system triangulates any newly clicked point in real time. We constrain the implementation to pure NumPy with no heavyweight ML framework so that the pipeline is reproducible from a single ~ 1000 -line script and trains in well under a second on a CPU.

This fits both the *Monocular Depth & Scale Recovery* and *Structure from Motion / 3D Reconstruction* topics from the syllabus, with the twist that absolute scale is recovered by user-supplied known distances rather than by monocular cues.

2 Baseline: Classical Pinhole Triangulation

The classical two-view triangulator treats each pixel as a 3-D ray and solves a skew-line intersection in least-squares form. We summarize the derivation in this section because the same OLS solve is reused inside our γ -grid method (§3).

2.1 Geometry of two camera rays

Camera 1 is located at a 3-D point $P_1 = [x_1, y_1, z_1]^\top$ with a normalized ray direction $\vec{d}_1 = [u_1, v_1, w_1]^\top$; camera 2 likewise with P_2 and $\vec{d}_2$. A point at distance t_i along ray i is

$$L_1(t_1) = P_1 + t_1\vec{d}_1, \quad L_2(t_2) = P_2 + t_2\vec{d}_2.$$

If the lines intersect, $P_1 + t_1\vec{d}_1 = P_2 + t_2\vec{d}_2$, which rearranges to

$$t_1\vec{d}_1 - t_2\vec{d}_2 = P_2 - P_1.$$

2.2 The matrix-vector equation $Ax = b$

Expanding component-wise yields a linear system

$$\underbrace{\begin{bmatrix} u_1 & -u_2 \\ v_1 & -v_2 \\ w_1 & -w_2 \end{bmatrix}}_A \underbrace{\begin{bmatrix} t_1 \\ t_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \end{bmatrix}}_b.$$

A has 3 rows and 2 columns: overdetermined. Pixel noise guarantees the rays will not intersect exactly, so no exact solution exists.

2.3 L2-norm error and OLS solution

Define the residual $e = Ax - b$ and minimize $\|e\|_2^2$:

$$E(x) = (Ax - b)^\top (Ax - b) = x^\top A^\top Ax - 2b^\top Ax + b^\top b.$$

This is a convex quadratic with gradient

$$\nabla E(x) = 2A^\top Ax - 2A^\top b.$$

Setting $\nabla E = 0$ yields the normal equations and the closed-form OLS solution

$$A^\top Ax = A^\top b \implies x = (A^\top A)^{-1} A^\top b. \quad (1)$$

The optimal (t_1, t_2) are obtained in one step; for educational reference we note that the same minimum is reachable by gradient descent $x_{k+1} = x_k - \alpha(2A^\top Ax_k - 2A^\top b)$, which converges because E is convex.

2.4 Final 3-D point and error distance

With optimal (t_1, t_2) , the closest points on each ray are

$$P_1^* = P_1 + t_1\vec{d}_1, \quad P_2^* = P_2 + t_2\vec{d}_2.$$

Because the lines are skew, the points do not coincide; the best estimate of the true 3-D object is the midpoint and the reconstruction error is the gap between the two:

$$\hat{P} = \frac{1}{2}(P_1^* + P_2^*), \quad \epsilon = \|P_1^* - P_2^*\|_2.$$

2.5 Why the pinhole baseline is insufficient

The OLS triangulator is mathematically clean but assumes (i) perfect pinhole optics with no radial or tangential distortion, (ii) a single focal length per camera, and (iii) *known* intrinsics. For a consumer stereo rig with unknown intrinsics we can still fit a single scalar focal length per camera plus the baseline (d_x, d_y) , but this 4-parameter model leaves systematic bias at the image edges where lens distortion dominates. Our experiments (§5) confirm this: the pinhole baseline achieves 0.180m mean absolute error on near-image points where geometry dominates but degrades to 0.448m on far points where distortion matters more.

3 Methodology: Anisotropic γ -Grid Stereo

We replace the single focal length per camera with a learnable 3×3 grid of inverse focal lengths, separately for the x and y image axes. The grid is bilinearly interpolated to produce a smooth per-pixel distortion correction without any polynomial distortion coefficients. Triangulation itself is unchanged from §2 — we modify *which rays we feed in*, not *how we triangulate them*.

3.1 Pixel and normalized coordinates

Working directly in pixel coordinates (u, v) is numerically unstable for optimization. We normalize relative to the optical center $(C_X, C_Y) = (W/2, H/2)$:

$$u_n = \frac{u - C_X}{C_X}, \quad v_n = \frac{v - C_Y}{C_Y}. \quad (2)$$

Normalized coordinates are centered at 0 and generally fall in $[-1, 1]$, decoupling the image resolution from the optimization space.

3.2 Ray equation

A pixel (u, v) defines a 3-D ray shot from the camera’s optical center. Our ray model is

$$r = \begin{bmatrix} u_n \gamma_x \\ v_n \gamma_y \\ 1 \end{bmatrix}, \quad (3)$$

where γ_x and γ_y are effective inverse focal lengths (in a perfect pinhole camera $\gamma_x = C_X/f_x$). By letting γ *vary across the image* we inherently model lens distortion without any polynomial distortion coefficients.

3.3 Bilinear γ interpolation

To make $\gamma(u, v)$ smooth and cheap we tile the image with a 3×3 grid of trainable nodes g_{ij} (cell dimensions $\text{CELL_W} = W/2$, $\text{CELL_H} = H/2$). Mapping (u, v) to continuous grid coordinates

$$f_x = u/\text{CELL_W}, \quad f_y = v/\text{CELL_H}, \quad (4)$$

we identify the top-left integer node (i_x, i_y) and the fractional distances $t_x = f_x - i_x$, $t_y = f_y - i_y$. The interpolated value is the weighted sum of the four surrounding nodes:

$$g(u, v) = (1 - t_x)(1 - t_y)g_{00} + t_x(1 - t_y)g_{10} + (1 - t_x)t_y g_{01} + t_x t_y g_{11}. \quad (5)$$

This guarantees C^0 continuity, costs four multiplications per query, and is $O(1)$.

3.4 Anisotropic grid: $\gamma_x \neq \gamma_y$

Traditional pinhole models assume one focal length per camera and symmetric radial distortion. We maintain two grids per camera, one for γ_x and one for γ_y , yielding a parameter tensor Γ of shape $(R, C, 2)$. The motivation is threefold:

1. **Sensor non-uniformity:** pixels are rarely square, so $f_x \neq f_y$.
2. **Asymmetric distortion:** manufacturing defects in lenses are not perfectly circular.
3. **Decoupled gradients:** γ_x strictly handles horizontal disparity (the depth signal); γ_y strictly handles epipolar Y-alignment.

3.5 Stereo geometry and triangulation

For a matched click pair (u_L, v_L) in the left image and (u_R, v_R) in the right, equation (3) produces two rays v_L and v_R . Let Z be the depth along the left camera's axis and Z_R the depth along the right; with baseline $d = [d_x, d_y, 0]^\top$ the geometric constraint $P_L = d + P_R$ becomes

$$Zv_L - Z_Rv_R = d \implies \underbrace{\begin{bmatrix} | & | \\ v_L & -v_R \\ | & | \end{bmatrix}}_A \underbrace{\begin{bmatrix} Z \\ Z_R \end{bmatrix}}_x = d. \quad (6)$$

The overdetermined system is solved by the same OLS step from (1):

$$x = (A^\top A)^{-1} A^\top d, \quad Z = x_1. \quad (7)$$

3.6 Disparity

In our ray space, horizontal disparity is $\text{disp} = v_{Lx} - v_{Rx}$. When $Z \gg d_x$ we have $Z \approx Z_R$ and equation (7) gives the well-known inverse relation

$$\text{disp} \approx \frac{d_x}{Z}. \quad (8)$$

This explains why stereo is precise at short range (large disparity) and imprecise at long range (disparity $\rightarrow 0$).

3.7 Loss design

A naive $L = (Z_{\text{pred}} - Z_{\text{true}})^2$ is unstable because $Z \propto 1/\text{disp}$: a one-pixel error at 10 m produces an order of magnitude larger gradient than at 1 m. We therefore optimize in ray-disparity space.

Relative disparity loss (L_{1D}). Define $\text{disp}_{\text{true}} = d_x/Z_{\text{true}}$ and the unit-less relative error

$$e_x = \frac{v_{Lx} - v_{Rx}}{\text{disp}_{\text{true}}} - 1 = \frac{(v_{Lx} - v_{Rx})Z_{\text{true}}}{d_x} - 1, \quad L_{1D} = \frac{1}{2}e_x^2. \quad (9)$$

This formulation treats a 5% depth error at 1 m identically to a 5% error at 10 m, stabilizing optimization.

Epipolar Y-alignment loss (L_Y). For perfect stereo the vertical ray components should match the physical vertical baseline:

$$e_y = (v_{Ly} - v_{Ry}) - \frac{d_y}{Z_{\text{true}}}, \quad L_Y = \frac{1}{2}e_y^2. \quad (10)$$

This forces the network to learn γ_y such that epipolar lines become horizontal in ray space.

Huber robustness. Human calibration clicks contain outliers. Standard MSE chases them; we wrap each error in a Huber loss with $\delta = 0.05$ (a 5% relative-error threshold):

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq \delta, \\ \delta(|e| - \frac{1}{2}\delta), & |e| > \delta, \end{cases} \quad \frac{\partial L_\delta}{\partial e} = \begin{cases} e, & |e| \leq \delta, \\ \delta \cdot \text{sign}(e), & |e| > \delta. \end{cases} \quad (11)$$

Beyond δ the gradient magnitude is capped at δ , bounding outlier influence.

3.8 Analytical gradients

We do not use automatic differentiation. All gradients are derived analytically using the chain rule. Let $D = v_{Lx} - v_{Rx}$. Inside the Huber quadratic region:

$$\frac{\partial L}{\partial D} = e_x \cdot \frac{Z_{\text{true}}}{d_x}, \quad \frac{\partial L}{\partial v_{Lx}} = +\frac{\partial L}{\partial D}, \quad \frac{\partial L}{\partial v_{Rx}} = -\frac{\partial L}{\partial D}, \quad (12)$$

$$\frac{\partial L}{\partial v_{Ly}} = e_y, \quad \frac{\partial L}{\partial v_{Ry}} = -e_y. \quad (13)$$

From (3), $v_{Lx} = u_{nL}\gamma_{xL}$, so $\partial v_{Lx}/\partial \gamma_{xL} = u_{nL}$; from (5), $\gamma_{xL} = \sum_{k=1}^4 w_k g_k$ where w_k are the bilinear weights. Composing,

$$\frac{\partial L}{\partial g_k} = \frac{\partial L}{\partial v_{Lx}} \cdot u_{nL} \cdot w_k. \quad (14)$$

Gradients with respect to the baseline are

$$\frac{\partial L}{\partial d_x} = e_x \cdot \left(-\frac{D Z_{\text{true}}}{d_x^2} \right), \quad \frac{\partial L}{\partial d_y} = e_y \cdot \left(-\frac{1}{Z_{\text{true}}} \right). \quad (15)$$

Outside the Huber quadratic region, every $e_\bullet$ in the equations above is replaced by $\delta \cdot \text{sign}(e_\bullet)$. The exact logic above is implemented in `forward_grads(...)` in `stereo_app.py`.

3.9 Adam optimizer

For any parameter $\theta \in \{\Gamma_L, \Gamma_R, d\}$ with gradient g_t at step t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (16)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t), \quad (17)$$

$$\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon). \quad (18)$$

We use $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Different parameters get different learning rates because they live on vastly different scales: $\eta_\Gamma = 10^{-1}$ (dimensionless, ~ 0.45) and $\eta_d = 10^{-3}$ (meters, ~ 0.1). A `ReduceLROnPlateau` schedule halves all three rates after 800 epochs without improvement, and early stopping triggers after 3000 stalled epochs.

3.10 Regularization

With 36 grid parameters per camera (+ baseline) and only ~ 15 –40 calibration points, overfitting is a real risk. We add two regularization terms directly to the gradients.

Anchor prior. Pulls Γ toward its physical initial value $G_0 = C_X/F_{\text{init}}$:

$$\nabla_{\text{anchor}} = \lambda_a(\Gamma - G_0). \quad (19)$$

Unused grid cells therefore stay physically sensible rather than drifting.

Smoothness Laplacian. Adjacent nodes should be similar. The 4-neighbor discrete Laplacian is

$$\nabla^2 G_{i,j} \approx 4G_{i,j} - G_{i-1,j} - G_{i+1,j} - G_{i,j-1} - G_{i,j+1}, \quad (20)$$

and adding $\lambda_s \nabla^2 G$ to the gradient makes the grid behave like an elastic sheet. We use $\lambda_a = \lambda_s = 10^{-3}$.

3.11 Polynomial post-correction

After the γ -grid converges, slight systematic biases may remain due to unmodeled physics (lens thickness, sensor non-linearity). We absorb the residual by fitting a 1-D quadratic on the calibration set:

$$Z_{\text{true}} = a + b Z_{\text{pred}} + c Z_{\text{pred}}^2, \quad (21)$$

solved instantly by ordinary least squares on a Vandermonde matrix of Z_{pred} . This adds 3 parameters; on our data the fit is $(a, b, c) = (0.020, 0.944, 0.015)$.

3.12 Numerical stability safeguards

1. **Parallel rays:** if $|v_{Lx} - v_{Rx}| < 10^{-9}$ the depth is undefined; the code returns NaN.
2. **Near-singular $A^\top A$:** falls back to the 1-D disparity formula $Z = d_x/D$.
3. **Γ clipping:** values are clipped to $[0.05, 5.0]$, preventing negative or infinite effective focal lengths.
4. **Minimum baseline:** $d_x \geq 0.02$ m, preventing division by zero in (9).

4 Auxiliary Components

4.1 Fundamental matrix for click assistance

To help the user click matching points, the UI draws an epipolar line in the right image whenever the user clicks the left. The fundamental matrix F maps a left point x_L to a right line $\ell_R = Fx_L$. We construct F locally from the per-pixel γ values:

$$K_L^{-1} = \begin{bmatrix} \gamma_{xL}/C_X & 0 & -\gamma_{xL} \\ 0 & \gamma_{yL}/C_Y & -\gamma_{yL} \\ 0 & 0 & 1 \end{bmatrix}, \quad E = [d]_{\times} = \begin{bmatrix} 0 & -d_z & d_y \\ d_z & 0 & -d_x \\ -d_y & d_x & 0 \end{bmatrix}, \quad (22)$$

$$F = K_R^{-\top} E K_L^{-1}, \quad [a, b, c]^\top = F[u_L, v_L, 1]^\top, \quad (23)$$

yielding the epipolar line $au_R + bv_R + c = 0$. Because the right pixel is not yet known we approximate K_R^{-1} using the mean of Γ_R .

4.2 Zero-mean NCC matching

A 1-D search along the epipolar line picks the integer pixel with maximum zero-mean normalized cross-correlation between a left patch A and a right patch B :

$$\text{ZNCC}(A, B) = \frac{\sum_i (A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum_i (A_i - \bar{A})^2} \sqrt{\sum_i (B_i - \bar{B})^2}} \in [-1, 1]. \quad (24)$$

Mean subtraction gives brightness invariance; variance normalization gives contrast invariance. The peak is refined to sub-pixel precision by fitting a parabola to the best score and its two neighbors (s_{-1}, s_0, s_{+1}):

$$\Delta = \frac{s_{-1} - s_{+1}}{2(s_{-1} - 2s_0 + s_{+1})}, \quad u_{\text{refined}} = u_{\text{best}} + \Delta. \quad (25)$$

5 Experiments

5.1 Dataset and protocol

We collected 43 hand-clicked stereo pairs across six scenes with ground-truth distances measured by tape. Z range: 1.35–8.75 m; 12 near points ($Z < 3$ m) and 31 far points ($Z \geq 3$ m). Image resolution is 2592×1944 . We compare three configurations:

- A. Pinhole OLS baseline** — one scalar γ per camera plus (d_x, d_y) : 4 parameters. Trained with the same loss as our full method, so the comparison isolates model capacity rather than loss design.
- B. γ -grid only** — anisotropic $3 \times 3 \times 2$ grid per camera plus baseline: 38 parameters.
- C. γ -grid + post-correction** — B plus the 3-parameter quadratic.

We report (i) *in-sample fit* — train on all 43, evaluate on the same 43 — and (ii) *leave-one-out cross-validation*: 43 folds of train-on-42, test-on-1. LOO is the honest generalization estimator at this dataset size. Metrics: mean absolute error (MAE, m), mean absolute percentage error (MAPE, %), maximum absolute error (m), plus a near/far MAE breakdown at the 3 m boundary.

5.2 Results

Table 1: In-sample fit (trained and evaluated on the same 43 points).

Configuration	MAE (m)	MAPE (%)	Max (m)	Near MAE	Far MAE
A) Pinhole OLS	0.373	8.39	2.487	0.181	0.447
B) γ -grid	0.145	3.90	0.694	0.119	0.156
C) γ -grid + post-corr.	0.116	3.46	0.545	0.115	0.117

5.3 Discussion

The γ -grid captures real distortion structure. On the in-sample fit it reduces MAE $2.6\times$ over the pinhole baseline ($0.373 \rightarrow 0.145$ m) and the maximum error drops $3.6\times$ ($2.49 \rightarrow 0.69$ m). The drop is consistent across the Z range, indicating the per-pixel anisotropic distortion model is

Table 2: Leave-one-out cross-validation ($n = 43$).

Configuration	MAE (m)	MAPE (%)	Max (m)	Near MAE	Far MAE
A) Pinhole OLS	0.373	8.36	2.563	0.180	0.448
B) γ -grid	0.300	9.04	1.792	0.373	0.271
C) γ -grid + post-corr.	0.269	8.62	1.854	0.372	0.229

doing genuine work, not just fitting noise. The saved calibration shows γ_x values ranging from 0.28 to 0.54 across the grid — a 90% spread — confirming the grid did not collapse to a constant. The recovered baseline $d_x = 0.233$ m and $d_y = 0.0005$ m matches the physical camera mount within sub-millimeter tolerance.

Overfitting is also real. The LOO MAE doubles from 0.145 m (in-sample) to 0.300 m, the expected price of 38 parameters fit on ~ 42 points per fold even with our anchor and Laplacian regularization. The pinhole baseline is rock-stable across the in-sample/LOO split ($0.373 \rightarrow 0.373$) because 4 parameters cannot overfit at this scale — this makes the in-sample/LOO gap the cleanest evidence of γ -grid overfitting.

Near versus far reveals a mirror tradeoff. The most interesting analytical finding from Table 2 is that the two methods are opposites:

- **Pinhole is good near, bad far** (0.180 m near vs 0.448 m far). Near points have large disparities, so the geometry dominates the distortion error.
- **γ -grid is bad near, good far** (0.373 m near vs 0.271 m far). The grid spends its capacity on far-point distortion correction where it matters most, and over-fits the near regime.
- Post-correction improves far-point MAE further ($0.271 \rightarrow 0.229$ m) but cannot rescue near points — a single global quadratic cannot undo overfitting that is locally heterogeneous.

This suggests two principled mitigations not pursued in this work: stronger λ_{anchor} in the near-image region, or reducing the grid to 2×2 and relying on the post-correction quadratic to capture remaining spatial variation.

6 Comparison with Existing Methods

Method	Calibration input	Distortion model	Output	Library
Zhang 2000 (<code>cv2.stereoCalibrate</code>)	~ 20 checkerboard images	k_1, k_2, k_3, p_1, p_2	full intrinsics + extrinsics	OpenCV
OpenCV SGBM	requires prior K, R, t	uses given K	dense disparity map	OpenCV
Hartley & Zisserman MVG, ch. 12	known fundamental matrix	none	sparse triangulation	—
Ours	≥ 15 in-scene clicks at known Z	learned $3 \times 3 \times 2$ γ -grid	sparse depth + post-corr	pure NumPy

The honest comparison: we lose the dense disparity map that SGBM produces and we require ground-truth distances during calibration. We gain target-free deployment, an order-of-magnitude

lower parameter count than Zhang’s intrinsic + distortion model, and an anisotropic distortion model that is in principle more general than 1-D radial. Our framework also produces an explicit fundamental matrix per click via $F = K_R^{-\top}[d]_{\times}K_L^{-1}$, which we use to draw the epipolar line in the right image and run ZNCC sub-pixel matching for click-assist.

7 Failure Cases and Limitations

1. **Parallel rays.** When $|v_{Lx} - v_{Rx}| < 10^{-9}$ the triangulation system is singular and we return NaN. Happens for very distant or near-coplanar viewing geometry.
2. **Disparity collapse at long range.** Beyond ~ 10 m on our rig the disparity approaches the click-noise floor and depth uncertainty grows as Z^2 regardless of model capacity.
3. **Near-region overfitting.** Documented in Table 2 and §5. Acceptable for a click-based measurement tool but problematic for dense reconstruction.
4. **Outlier clicks survive Huber.** A click on the wrong object cannot be rejected by any robust loss — only by RANSAC-style consensus, which we did not implement.

8 Conclusion

We presented a stereo depth pipeline that replaces classical pinhole intrinsics with a trainable anisotropic γ -grid, calibrated from a handful of in-scene clicks at known distances. The method reduces leave-one-out mean absolute depth error from 0.373 m to 0.269 m versus a pinhole-OLS baseline on tape-measured points in 1.35–8.75 m range, with the largest gains on far points where distortion matters most. The same experiments expose an overfitting gap and a near/far tradeoff that suggest concrete next steps.

Takeaway: don’t calibrate the lens — calibrate the rays.

Appendix A. Pipeline at a Glance

1. **Pixel click:** user selects (u_L, v_L) in the left image.
2. **Ray generation:** bilinear γ lookup $\rightarrow$ ray v_L via (3).
3. **Epipolar search:** build local fundamental matrix F ; ZNCC along the line in the right image $\rightarrow (u_R, v_R)$ with sub-pixel refinement.
4. **Triangulation:** OLS solve (7) $\rightarrow$ raw Z .
5. **Correction:** quadratic $Z_{\text{true}} = a + bZ + cZ^2 \rightarrow$ final physical distance.

Appendix B. Computational Complexity

- Bilinear interpolation: $O(1)$ per point, 4 multiplications.
- Triangulation: $O(1)$ via a 2×2 matrix solve.

- Training: $O(N \cdot E)$ for N points and E epochs. With $N \approx 40$ and $E \leq 20000$, total operations are $\sim 10^6$, sub-second on a modern CPU.
- ZNCC matching: $O(S \cdot P^2)$ where $S \approx 300$ px is the search range and $P = 15$ the patch size $\approx 67,000$ ops per click.

Appendix C. Function $\leftrightarrow$ Math Mapping

- `bilinear(u, v, Gamma)` $\leftrightarrow$ Equation (5).
- `build_ray(u, v, Gamma)` $\leftrightarrow$ Equations (2) and (3).
- `triangulate(...)` $\leftrightarrow$ Equation (7): builds A and calls `np.linalg.solve(AtA, Atd)`.
- `forward_grads(...)` $\leftrightarrow$ Equations (9)–(14): relative-disparity loss, Huber clipping, chain rule to Γ_L, Γ_R, d .
- `train(...)` $\leftrightarrow$ Adam optimizer and regularization.
- `fundamental_matrix_local(...)` $\leftrightarrow K_L^{-1}, E, F$.
- `epipolar_guided_match(...)` $\leftrightarrow$ ZNCC and parabolic refinement.
- `fit_postcorrection(...)` $\leftrightarrow$ Vandermonde solve for (a, b, c) .

Reproducibility

- Code: `stereo_app.py` (full system), `experiments/run_experiments.py` (this study).
- Data: `calibration_points.json` (43 clicks), `stereo_calibration.json` (saved γ -grid).
- Math reference: `docs/baseline_classical_stereo.tex`, `docs/final_gamma_grid_method.tex`.
- Raw numerical results: `experiments/results.json`.
- Seed: 0 (numpy default RNG and `VAL_SEED = 0`); LOO is deterministic.